Durham University

Lecture 7: Functors

COMP2221: Functional programming

Maximilien Gadouleau

m.r.gadouleau@durham.ac.uk

Higher Order Functions

Higher order functions

- We've seen many functions that are naturally recursive
- We'll now look at *higher order functions* in the standard library that capture many of these patterns

Definition (Higher order function)

A function that does at least one of

- take one or more functions as arguments
- returns a function as its result

- Due to currying, every function of more than one argument is higher-order in Haskell

```
add :: Num a => a -> a -> a
add x y = x + y

-- "add 1" returns a function!
Prelude> :type add 1
Num a => a -> a
```

Examples for higher order functions on lists

- Many *linear recursive* functions on lists can be written using higher order library functions
- `map`: apply a function to all elements in a list

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f xs = [f x | x <- xs]
```

- `filter`: select elements from a list that satisfy a predicate

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p xs = [x | x <- xs, p x]
```

- `any`, `all`, `foldr`, `takeWhile`, `dropWhile`, ...
- For more, see `http:`

  `//hackage.haskell.org/package/base-4.12.0.0/docs/Prelude.html#g:13`

Function composition

- Often tedious to write brackets and explicit variable names
- Can use *function composition* to simplify this

$$(f \circ g)(x) = f(g(x))$$

- Haskell uses the `(.)` operator

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
f . g = \x -> f (g x)
-- example
odd a = not (even a)
odd   = not . even -- Point-free style: no need for the variable a
```

- Useful for writing composition of functions to be passed to other higher order functions
- Removes need to write λ-expressions
- Called "pointfree" style.

- Saw example higher-order functions on lists
- Now we'll get even more generic and implement these generic patterns for custom datatypes

Functors

Use type classes to implement generic higher order functions

- Recall, Haskell has a concept of *type classes*
- These describe interfaces that can be used to constrain the polymorphism of functions to those types satisfying the interface

Example

- `(+)` acts on any type, as long as that type implements the `Num` interface

```
(+) :: Num a => a -> a -> a
```

- `(<)` acts on any type, as long as that type implements the `Ord` interface

```
(<) :: Ord a => a -> a -> Bool
```

- Haskell comes with *many* such type classes encapsulating common patterns
- When we implement our own data types, we can "just" implement appropriate instances of these classes

Use type classes to implement generic higher order functions

- Recall, Haskell has a concept of *type classes*
- These describe interfaces that can be used to constrain the polymorphism of functions to those types satisfying the interface
- Haskell has *many* type classes encapsulating common patterns in the standard library:
  - `Num`: numeric types
  - `Eq`: equality types
  - `Ord`: orderable types
  - `Functor`: mappable types
  - `Foldable`: foldable types
  - ...
- If you implement a new data type, it is worthwhile thinking if it satisfies any of these interfaces
- When we implement our own data types, we can "just" implement appropriate instances of these classes

Let's look at the types of two "map" functions

```
data [] a = [] | a:[a]
map :: (a -> b) -> [a] -> [b]

data BinaryTree a = Leaf a | Node a (BinaryTree a) (BinaryTree a)
bmap :: (a -> b) -> BinaryTree a -> BinaryTree b
```

Only difference is the type name of the container. This suggests that we should make a "Container" type class to capture this pattern.

Haskell calls this type class `Functor`

```
class Functor c where
  fmap :: (a -> b) -> c a -> c b
```

If a type implements the `Functor` interface, it defines a data structure that we can transform the elements of in a systematic way.

`fmap`: a generic map function

```
Prelude> :t fmap
fmap :: Functor f => (a -> b) -> f a -> f b
Prelude> fmap (*2) [1, 2, 3]
[2, 4, 6]

class Functor f where
   fmap :: (a -> b) -> f a -> f b
```

- Works on any mappable structure
- Must obey *functor laws*:
- `fmap id c == c` Mapping the identity function over a structure should return the structure untouched.
- `fmap f (fmap g c) == fmap (f . g) c` Mapping over a container should distribute over function composition (since the structure is unchanged, it shouldn't matter whether we do this in two passes or one).

Instance declaration for Functors

Use an *instance* declaration to attach an `fmap` implementation to a container type.

```
data List a = Nil | Cons a (List a)
  deriving (Eq, Show)

instance Functor List where
  fmap _ Nil = Nil
  fmap f (Cons a tail) = Cons (f a) (fmap f tail)

data BinaryTree a = Leaf a | Node a (BinaryTree a) (BinaryTree a)
  deriving (Eq, Show)

instance Functor BinaryTree where
  fmap f (Leaf a) = Leaf (f a)
  fmap f (Node a l r) = Node (f a) (fmap f l) (fmap f r)
```

Generic code

```
list = Cons 1 (Cons 2 (Cons 4 Nil))
btree = Node 1 (Leaf 2) (Leaf 4)

-- Generic add1
add1 :: (Functor c, Num a) => c a -> c a
add1 = fmap (+1)

Prelude> add1 list
Cons 2 (Cons 3 (Cons 5 Nil))
Prelude> add1 btree
Node 2 (Leaf 3) (Leaf 5)
```

Are all containers Functors?

- It seems like any type that takes a parameter might be a `Functor`
- This is not necessarily the case, we require more than just type-correctness

```haskell
-- A type describing functions from a type to itself
data Fun a = MakeFunction (a -> a)

instance Functor Fun where
 fmap f (MakeFunction g) = MakeFunction id
```

This code type-checks `id :: a -> a` but does not obey the *Functor laws*

1. `fmap id c == c`
2. `fmap f (fmap g c) == fmap (f . g) c`

How many definitions?

- If I come up with a definition of `fmap` for a type, might there have been another one?
- No! if you can confirm that the functor laws hold

```
fmap id == id
fmap (f . g) == fmap f . fmap g
```

- then you must have written the right thing!

Correctness of `listMap`

```haskell
data List a = Nil | Cons a (List a) deriving (Eq, Show)

instance Functor List where
  fmap _ Nil = Nil
  fmap f (Cons x xs) = Cons (f x) (fmap f xs)
```

To show `fmap id == id`, need to show `fmap id (Cons x xs) == Cons x xs` for any x, xs.

```haskell
-- Induction hypothesis
fmap id xs = xs
-- Base case
-- apply definition
fmap id Nil = Nil
-- Inductive case
fmap id (Cons x xs) = Cons (id x) (fmap id xs)
== Cons x (fmap id xs)
== Cons x xs -- Done!
```

Exercise: check whether the second law holds